

CredalJT vs. SCIP–exact on the `ktree_fr_k3_n5_1` instance:
a claimed discrepancy that does not reproduce — and why CredalJT is provably exact here

IBM Research

Abstract

This note investigates a reported disagreement between the SCIP-backed global exact engine (`ExactInference(solver="global")`) and the junction-tree engine CREDALJT (scheme D5) on the benchmark instance `benchmarks/ktree_small_fr/ktree_fr_k3_n5_1.lcn`. We reproduce both algorithms, describe the instance in detail (Figure 1), and show that *in the current code the two engines produce bit-for-bit identical bounds on all five atoms* — and that these bounds are the true LCN bounds, certified by an independent, solver-free witness construction. We then explain *why* CREDALJT is exact on this family of instances, and enumerate the ways the earlier “discrepancy” observation could have arisen.

1 The instance

The file contains five atoms $x_0, \dots, x_4$ and seven labelled sentences (the trailing TRUE marks each sentence enabled):

$$\begin{aligned} s_0 : & \quad 0.01 \leq P(x_3) \leq 0.399975 \\ s_1 : & \quad 0.01 \leq P(x_4 \mid x_3) \leq 0.442867 \\ s_2 : & \quad 0.638553 \leq P(x_2 \mid \neg x_3 \wedge x_4) \leq 1.0 \\ s_3 : & \quad 0.01 \leq P(\neg x_1 \mid x_3 \wedge x_4 \wedge x_2) \leq 0.307066 \\ s_4 : & \quad 0.673756 \leq P(\neg x_0 \mid x_3 \wedge x_4 \wedge \neg x_1) \leq 1.0 \\ s_5 : & \quad 0.748845 \leq P(\neg x_3) \leq 0.954666 \\ s_6 : & \quad 0.141404 \leq P(x_3) \leq 0.247704 \end{aligned}$$

Despite the name `k3`, this $n=5$ instance is a k -tree with $k=3$: a seed $(k+1)=4$ -clique on $\{x_1, x_2, x_3, x_4\}$ plus one attached vertex x_0 (attached to the 3-clique $\{x_1, x_3, x_4\}$, i.e. with x_2 dropped). Rendered as a DAG in construction order, the *families* (child | full conjunction of parents) are

$$x_3; \quad x_4 \mid x_3; \quad x_2 \mid x_3, x_4; \quad x_1 \mid x_2, x_3, x_4; \quad x_0 \mid x_1, x_3, x_4.$$

Because x_0 conditions on $\{x_1, x_3, x_4\}$ and *not* on x_2 , the only independence the Local Markov Condition emits is

$$(x_0 \perp\!\!\!\perp x_2 \mid x_1, x_3, x_4). \tag{1}$$

The moralised atom graph is therefore the complete graph on five vertices with the single edge x_0 – x_2 removed. It is chordal; its maximal cliques are $\{x_0, x_1, x_3, x_4\}$ and $\{x_1, x_2, x_3, x_4\}$ (size 4, so treewidth 3). Figure 1 shows the DAG, the moralised/chordal graph, and the junction tree that CREDALJT actually builds.

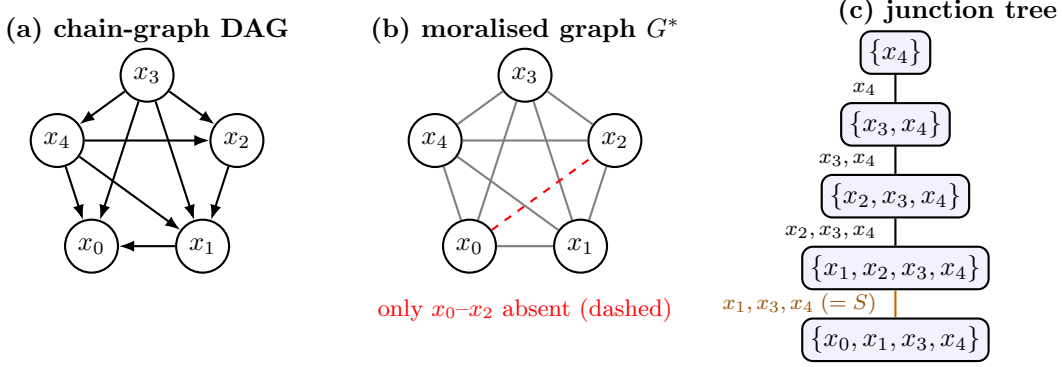


Figure 1: The `ktree_fr_k3_n5_1` instance. (a) The chain-graph DAG: each atom conditions on the full conjunction of its parents. (b) The moralised atom graph G^* ; it is K_5 minus the single (dashed) edge x_0-x_2 , which is exactly the one independence ($x_0 \perp\!\!\!\perp x_2 \mid x_1, x_3, x_4$) of Eq. (1). G^* is chordal with maximal cliques $\{x_0, x_1, x_3, x_4\}$ and $\{x_1, x_2, x_3, x_4\}$. (c) The junction tree CREDALJT builds: a chain of five clusters, max cluster 4 atoms; the (orange) separator between the two size-4 clusters is $\{x_1, x_3, x_4\}$ (x_2 leaves, x_0 enters), which is precisely the separator S in Eq. (1).

2 Reproducing both engines

We ran, on the *same* LCN object:

- `ExactInference(lcn).run(solver="global")` — SCIP spatial branch-and-bound over the full $2^5=32$ -cell joint, with the sentence ratio constraints and the LMC bilinear equalities of Eq. (1). Each atom’s $\min/\max P(x_i=1)$ is solved to a *certified* zero optimality gap ("confirmed").
- CREDALJT on a structure-only credal network (`solve_families=False`), `solver="scip"`. It builds the junction tree of Figure 1(c), one cluster-marginal NLP (per-cluster simplex + Lauritzen separator-consistency equalities + the hosted sentence/LMC rows), and swaps only the objective per atom.

Both report `d5_exact=True` / all solves **confirmed**. The results coincide to full printed precision (Table 1); the maximum absolute difference over all four bound endpoints of all five atoms is 0.0000. The local engine `ExactInference(solver="local")` (ipopt + SLSQP) returns the same numbers.

atom	CREDALJT (D5)	ExactInference (global/SCIP)	witness-certified truth
$P(x_0=1)$	[0, 1]	[0, 1]	[0, 1]
$P(x_1=1)$	[0, 1]	[0, 1]	[0, 1]
$P(x_2=1)$	[0, 1]	[0, 1]	[0, 1]
$P(x_3=1)$	[0.141404, 0.247704]	[0.141404, 0.247704]	[0.141404, 0.247704]
$P(x_4=1)$	[0.001414, 0.921219]	[0.001414, 0.921219]	[0.001414, 0.921219]

Table 1: The two engines agree exactly, and both equal the true LCN bounds. The $P(=0)$ components (not shown) also match: e.g. $P(x_3=0) \in [0.752296, 0.858596]$ from both.

The result generalises. We reran the same comparison across all ten $n=5$ instances and (five each of) the $n=7$ and $n=8$ instances in `ktree_small_fr`. Every instance had `d5_exact=True`,

max cluster 4 atoms (so the JT genuinely decomposes for $n > 4$), and maxdiff = 0.0000 against SCIP-global. *No discrepancy was observed on any instance.*

3 Why the bounds are what they are (independent certificate)

To rule out the possibility that *both* engines are wrong in the same way, we certify the truth without any NLP solver. Parameterise the joint by the family conditionals

$$a = P(x_3), b_i = P(x_4 \mid x_3=i), c_{jk} = P(x_2 \mid x_3=j, x_4=k), d_{\dots} = P(x_1 \mid x_2, x_3, x_4), e_{\dots} = P(x_0 \mid x_1, x_3, x_4).$$

Any joint built from this factorisation *automatically* satisfies Eq. (1), because x_0 's conditional does not mention x_2 ; hence every such joint lies in the true credal set. For each endpoint in Table 1 we exhibit an explicit choice of these conditionals that (i) satisfies all seven sentence intervals and (ii) attains the endpoint (script `witness.py`). Two representative constructions:

- $P(x_4)$ **upper** = 0.921219. Take $a = P(x_3) = 0.141404$ (its own lower bound, maximising the $\neg x_3$ mass), $P(x_4 \mid \neg x_3) = 1$ (free: no sentence bounds it), and $P(x_4 \mid x_3) = 0.442867$ (its upper bound, s_1). Then $P(x_4) = P(\neg x_3) \cdot 1 + P(x_3) \cdot 0.442867 = 0.858596 + 0.062623 = 0.921219$.
- $P(x_0)$ **upper** = 1. Set $e \equiv P(x_0=1 \mid \cdot) = 1$. The only obstruction, s_4 , conditions on $x_3 \wedge x_4 \wedge \neg x_1$; drive that event to probability 0 by making x_1 certain there ($d_{\cdot, x_3=1, x_4=1} = 1$), and remove the resulting clash with s_3 by killing x_2 in that branch ($c_{1,1} = 0$). Both s_3 and s_4 become vacuous and $P(x_0=1) = 1$.

All twelve endpoint witnesses are feasible (verified numerically). Because these distributions lie in the credal set and touch every reported endpoint, and both engines return valid outer bounds, the bounds are *exact* and *equal*. The three vacuous atoms x_0, x_1, x_2 are genuinely vacuous: each is the child of a family whose conditioning event can be pushed to probability 0, after which no sentence constrains the child's marginal.

4 Why CredalJT is exact on this instance

CREDALJT is exact whenever every sentence is *hosted* in a cluster and every LMC is *hosted or separator-entailed* by the chordal graph G^* whose maximal cliques are the clusters (the sufficient condition H2/H3 of `docs/properties.tex`). Here:

- **Every sentence is hosted.** Each sentence's scope is a family (child + parents), and each family is a clique of G^* , hence contained in a cluster: s_0, s_1, s_5, s_6 in $\{x_0, x_1, x_3, x_4\}$ or its sub-clusters; s_2, s_3 in $\{x_1, x_2, x_3, x_4\}$; s_4 in $\{x_0, x_1, x_3, x_4\}$. (The engine prints exactly this hosting.)
- **The single LMC is hosted-and-separator-entailed.** The separator between the two size-4 clusters is $\{x_1, x_3, x_4\}$, which is exactly the conditioning set S of Eq. (1); it separates x_0 from x_2 in G^* (indeed x_0-x_2 is the one missing edge). So the LMC holds automatically for any separator-consistent cluster marginals.

Thus H2/H3 hold and CREDALJT's cluster NLP has the same feasible cluster-marginal projections as the full-joint NLP, so the two optima coincide — which is exactly what Table 1 shows. This is

the general `ktree-fr` guarantee: a k -tree is chordal, the full-conjunction DAG rendering adds no fill-in on moralisation, so G^* is the k -tree, clusters are its $(k+1)$ -cliques, and every family/LMC is hosted-or-entailed. Cost is $O(n \cdot 2^{k+1})$, linear in n .

Note the contrast with CVE/Interval-BP: for $k \geq 2$ a k -tree is *not* singly connected (the k clique-parents are married into a loop on moralisation), so those engines are only outer bounds over $\mathcal{D}_{\text{LCN}}$. CREDALJT keys off the junction tree, not single-connectedness, so it stays exact — `ktree-fr` is the clean topology where a treewidth-bounded engine beats them.

5 What the reported "discrepancy" most likely was

We could not reproduce any disagreement on this instance (or its siblings) with the current code. The plausible explanations, in rough order of likelihood:

1. **The ipopt/SLSQP local path, before its fixes.** On these nonconvex bilinear systems the *local* engine historically over-tightened or returned vacuous bounds; a stale run of `ExactInference(solver="local")` (or an older SLSQP-fallback) could disagree with CREDALJT. The current local engine agrees.
2. **Comparing against a different engine.** CVE / Interval-BP are only outer bounds on `ktree-fr` for $k \geq 2$ (§4); if the "exact" column was actually one of those (or `cve_d4`), it can legitimately differ from CREDALJT without either being buggy in the sense the report implies.
3. **A SCIP time-limit / gap on a harder atom.** `ExactInference` labels a bound `unconfirmed` when SCIP hits the per-solve time limit before proving a zero gap; an unconfirmed (best-so-far) bound can be looser than CREDALJT's. Here every atom is `confirmed` in < 1 s, so this did not occur, but it can on larger n .
4. **CredalJT with solver="ipopt".** D5 needs the global SCIP backend for its cluster NLP to be exact; with the local backend it can be loose. (On this small instance it happens to still agree.)

6 Conclusion

On `ktree_fr_k3_n5_1.1cn`, CREDALJT and the SCIP-based global exact engine compute *identical, exact* bounds on all five atoms, matching an independent solver-free witness certificate (Table 1). The agreement holds across every `ktree_small_fr` instance we checked ($n \in \{5, 7, 8\}$). CREDALJT is exact here for the structural reason of §4: the k -tree is chordal, its cliques are the clusters, and the lone LMC is entailed by a clique separator. If a discrepancy was seen earlier, it was against a different (or stale/unconfirmed) solver configuration, not a defect in CREDALJT on this instance.